

PROYECTO FINAL · AI BOOTCAMP

SmartSpend: De Datos Heredados a Inteligencia de Gasto

Una plataforma de inteligencia de gastos del hogar construida con React, FastAPI, MySQL y Power BI — desde un fichero plano de MS Access hasta un esquema Snowflake normalizado con capacidades predictivas.



React 18



FastAPI



MySQL 8



Power BI



El Problema: Gastos Fragmentados sin Profundidad Analítica

La mayoría de las aplicaciones bancarias ofrecen únicamente el gasto mensual total, sin desgloses accionables. En un hogar compartido, los patrones de consumo individuales quedan completamente invisibles.

Seguimiento de Precios por Artículo

Imposible monitorizar cómo varía el precio de un producto específico (ej. plátanos) entre distintos supermercados a lo largo del tiempo.

Análisis Multi-Usuario

En hogares compartidos, resulta imposible separar y comparar el comportamiento de gasto individual (ej. Mum vs. Bunty).

Transparencia por Categoría

Categorías de alto coste como fisioterapia, suplementos o servicios quedan sistemáticamente subestimadas sin desglose estructurado.

Generación de Métricas

Ausencia de KPIs como gasto mensual total, categoría más cara, coste semanal medio de alimentación o distribución por establecimiento.

Misión del Proyecto: Transparencia Inteligente del Gasto

La Visión

SmartSpend es un sistema full-stack de inteligencia de gastos que recoge datos estructurados a través de una interfaz web, los almacena en una base de datos relacional normalizada y los expone para análisis avanzado en Power BI — sentando las bases para modelos predictivos con IA.

FASE ACTUAL: MVP COMPLETADO

Objetivos del MVP

- Seguimiento de precios al nivel de artículo individual
- Separación de gastos por usuario en el mismo hogar
- Desglose por categoría, marca y establecimiento
- Pipeline de datos listo para Power BI y para IA futura

7

Tablas relacionales

Esquema Snowflake totalmente
normalizado

18

Medidas DAX

Motor analítico de Power BI

6

Páginas de dashboard

Perspectivas analíticas interactivas

121+

Productos catalogados

Con categoría y marca normalizadas

El Stack Tecnológico: Cada Herramienta en su Lugar

Cada tecnología fue seleccionada con criterio técnico para maximizar la integridad de datos, la escalabilidad y la preparación para la analítica avanzada.



Frontend — React 18

SPA con layout dividido en pantalla, formulario dinámico de compra multi-artículo y componentes `react-select` con búsqueda y creación en línea. Gestión de estado limpia con `hooks` funcionales.



Backend — FastAPI + Pydantic v2

Framework Python de alto rendimiento ideal para arquitecturas basadas en API y futura integración con IA. Validación estricta de datos con Pydantic v2 y transacciones atómicas en el router de compras.



Base de datos — MySQL 8 + SQLAlchemy

Esquema relacional normalizado (Snowflake) con claves foráneas referenciadas, restricciones UNIQUE y modelos ORM que mapean 1:1 con las tablas SQL. Diseñado para consultas analíticas y compatibilidad con Power BI.

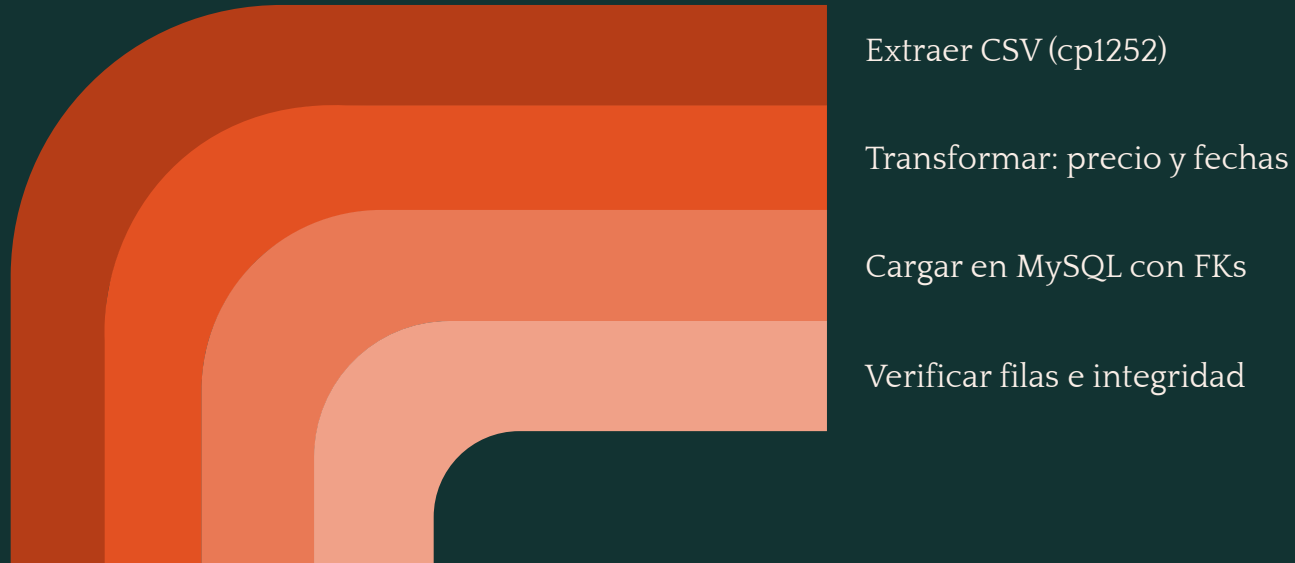


Analytics — Power BI

Herramienta BI estándar de la industria conectada directamente a MySQL. Aprovecha el esquema Snowflake para modelado fact/dimensión, compresión VertiPaq y filtrado dinámico con slicers independientes por categoría y marca.

El Reto de Migración: ETL desde MS Access

Los datos históricos del hogar vivían en una base de datos heredada de Microsoft Access — formato plano, sin normalizar, con codificación Windows cp1252 y formatos europeos de precio y fecha. Migrarlos al nuevo esquema MySQL requirió un pipeline ETL robusto construido en Python.



El mayor desafío fue la gestión simultánea de múltiples problemas de calidad de datos: precios europeos (€ 3,50 → float(3.50)), fechas en formato DD/MM/YYYY, restricciones de clave foránea durante TRUNCATE masivo y el mapeado de IDs padre-hijo para órdenes e ítems. El resultado: una migración idempotente y reproducible que transforma datos heredados en hechos analíticos limpios.

Codificación

`encoding="cp1252"` en `pd.read_csv()`

Decimales EU

Strip €, reemplazar `,` por `.`, cast a float

Fechas

`dayfirst=True` con `pd.to_datetime()`

Integridad FK

SET FOREIGN_KEY_CHECKS = 0/1 en bulk inserts

Excelencia en Backend: Transacciones Atómicas y Validación

Lógica de Transacción Atómica

El router de compras implementa la lógica de negocio más compleja del sistema. Tanto POST como PUT siguen el mismo patrón atómico: validar claves foráneas, crear/actualizar la cabecera, hacer `flush` para asegurar el ID, procesar cada línea de artículo, calcular subtotales y hacer `commit` atómico. Cualquier fallo activa `db.rollback()` completo.

- ❏ La estrategia «borrar y reinsertar» en PUT garantiza consistencia total: no quedan líneas huérfanas, el contrato con el frontend es simple y se evita lógica de merge compleja.

Pydantic v2 – Contratos de API

- **EntityCreate:** Enforce campos requeridos y coerción de tipos en POST
- **EntityUpdate:** Todos los campos opcionales con `exclude_unset=True` para actualizaciones parciales en PUT
- **EntityResponse:** Serialización ORM→JSON con `from_attributes=True`
- **PurchaseCreate:** Lista anidada de `PurchaseItemCreate` – un recibo completo en una sola petición HTTP

01

VALIDAR

Confirmar existencia de `user_id` y `shop_id`. HTTP 404 si no existen.

02

CABECERA

Crear/actualizar `Purchase`. `db.flush()` para asegurar el ID.

03

ÍTEMS

Validar `product_id`, calcular `subtotal = cantidad × precio`, insertar `PurchaseItem`.

04

TOTAL

Sumar subtotales → `purchase.total_amount`.

05

COMMIT

`db.commit()` atómico. Cualquier error → `db.rollback()` + `HTTPException`.

Frontend UX: Dashboard en Pantalla Dividida

La interfaz React implementa un diseño responsive de pantalla dividida: el formulario de transacción fijo a la izquierda y el gestor de dashboard dinámico a la derecha – ambos con scroll vertical independiente en pantallas $\geq 1024\text{px}$.

Panel Izquierdo – TransactionForm

- Selector de usuario, establecimiento y fecha
- Filas de artículos dinámicas con subtotal calculado en tiempo real
- Dropdowns `CreatableSelect`: búsqueda entre 121+ productos con creación en línea
- Creación de categorías al vuelo vía endpoint get-or-create
- Formato EUR consistente con `Intl.NumberFormat`
- Modo doble: creación (POST) y edición (PUT) según presencia de `editingTransaction`

Panel Derecho – DashboardManager (Grid Center)

- 👤 **Users** – CRUD completo con formulario inline y confirmación de borrado
- 🏪 **Shops** – Gestión de tiendas con nombre único forzado
- 🏦 **Transactions** – Historial completo con detalle por artículo; edición activa TransactionForm
- 📦 **Edit Products** – Sub-grid con Productos, Categorías y Marcas

Cada panel sigue un patrón uniforme: DataCard (grid clicable), PanelHeader con navegación y formularios inline para CRUD sin salir de la vista.

Arquitectura de Datos: El Esquema Snowflake

La base de datos MySQL implementa un **esquema Snowflake** — extensión del Star Schema donde las dimensiones están completamente normalizadas en sub-dimensiones. Sirve simultáneamente a la carga OLTP del backend FastAPI y a la analítica OLAP de Power BI.

Tablas de Hechos (Facts)

purchases

Cabecera de recibo: user_id FK, shop_id FK, fecha, descuento, coste de envío, total_amount

purchase_items

Línea de artículo: product_id FK, cantidad, precio unitario, subtotal — granularidad máxima

Justificación de la normalización

- Elimina redundancia: strings de categoría/marca almacenados una sola vez
- Integridad referencial via FKs con restricciones UNIQUE
- Tablas de dimensión pequeñas = mejor compresión VertiPaq en Power BI
- Slicers independientes por categoría y marca sin parsear atributos de producto

Tablas de Dimensiones

Tabla	Rol
users	Miembros del hogar (email UNIQUE)
shops	Establecimientos / vendedores (name UNIQUE)
products	Catálogo con category_id FK y brand_id FK
categories	Clasificación jerárquica de productos
brands	Sub-dimensión de marcas comerciales

Cadena de relaciones: users —< purchases —< purchase_items >— products >— categories >— brands / shops —< purchases

Arquitectura de Datos: Esquema Snowflake

Diagrama Entidad-Relación — Arquitectura Snowflake

Tablas de Hechos

- **Compras:** Transacciones maestras — fecha, usuario, tienda, importe total
- **Artículos de Compra:** Detalle línea — producto, cantidad, precio, subtotal

Tablas de Dimensiones

- **Usuarios · Tiendas · Marcas · Categorías**
- Normalización completa elimina redundancia

Integridad de Datos

- FK estrictas con integridad referencial
- Restricciones UNIQUE y reglas CASCADE

La Capa de Inteligencia: Power BI y 18 Medidas DAX

El dashboard de Power BI transforma datos transaccionales en métricas financieras accionables a través de 18 medidas DAX explícitas organizadas en seis categorías funcionales — conectado directamente a MySQL aprovechando el esquema Snowflake.

Vista Ejecutiva

Gasto total en el tiempo, desglose por categoría y top de establecimientos. Responde: ¿cómo evoluciona mi flujo financiero global?

Comparativa de Tiendas

Herramienta de seguimiento de precios: historial exacto de fechas, cantidades y precios para detectar cuándo un supermercado modifica sus tarifas.

Inteligencia Temporal

MoM Change %, Rolling 3M Avg, YTD Spend y gráfico Waterfall de varianza mensual positiva/negativa.

Categorías y Productos

Drill-down desde macrocategorías (Alimentación) hasta costes por artículo específico con granularidad de línea.

Análisis por Usuario

Huella financiera individual. Arquitectura escalable lista para separación de costes multi-usuario en el futuro.

Rastreador de Salud

Página aislada para bienestar físico y mental: inversión en salud YTD, contador de sesiones clínicas y registro cronológico completo.

Medidas de Inteligencia de Precios

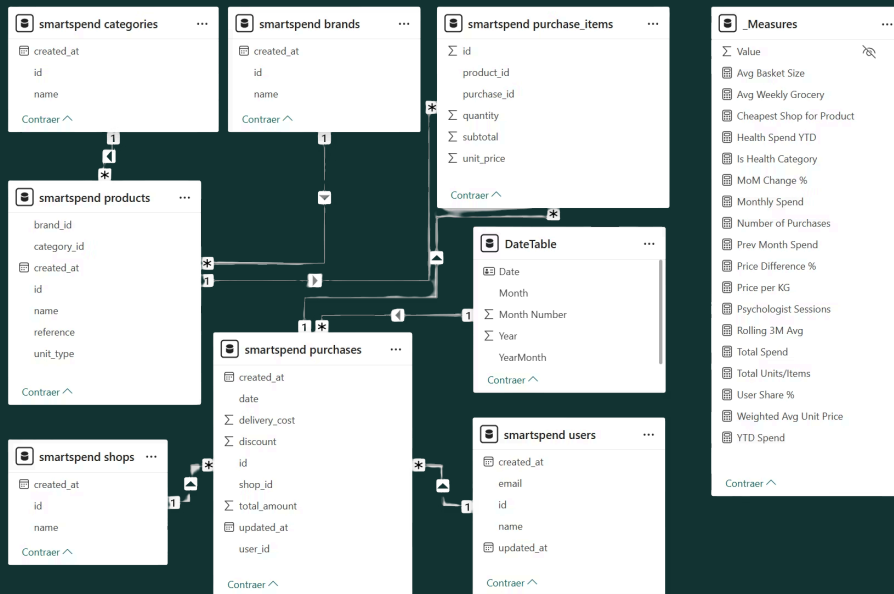
- **Weighted Avg Unit Price:** `DIVIDE(SUM(subtotal), SUM(quantity))` — coste medio real
- **Cheapest Shop for Product:** motor de texto dinámico con TOPN para identificar el comercio más barato
- **Price Difference %:** gap de inflación entre el establecimiento más caro y el más barato

Formato Condicional Dinámico

- **MoM Color (HEX):** rojo/verde según tendencia de gasto
- **Trend Arrow (Unicode):** indicador visual ↑ ↓ automático
- **Dynamic Title:** etiquetas contextuales adaptadas al filtro activo

Vista del Modelo en Power BI: Esquema Snowflake en VertiPaq

Esta vista representa la implementación real del esquema Snowflake dentro del motor VertiPaq de Power BI. El modelo conecta directamente a MySQL y replica la arquitectura normalizada, permitiendo slicing & dicing multidimensional con máxima eficiencia de compresión columnar.



Power BI Model View — Relaciones VertiPaq sobre esquema Snowflake normalizado

Relaciones 1:* (Uno a Varios)

- Cada relación sigue el patrón dimensional: la tabla de dimensión (lado 1) filtra la tabla de hechos (lado *)
- Garantiza integridad referencial durante el filtrado dinámico cruzado
- Propagación de filtros unidireccional desde dimensiones hacia hechos

Tablas del Modelo

- **Fact Tables:** purchases, purchase_items — grano transaccional
- **Dimensions:** users, shops, brands, categories, products
- **DateTable:** tabla de calendario generada para inteligencia temporal (Year, Month, YearMonth)
- **Measures:** tabla dedicada con 23 medidas DAX organizadas

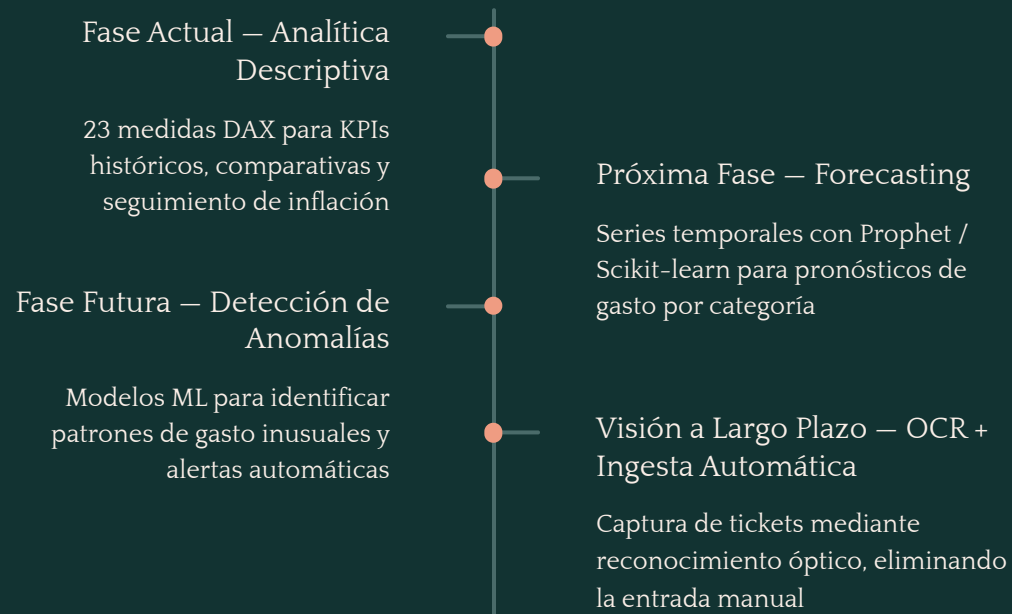
Optimización VertiPaq

- Compresión columnar optimizada por la normalización completa
- Cardinalidad reducida en dimensiones mejora el rendimiento de consultas
- Arquitectura lista para escalar sin degradación

IA, Demo y Lecciones Aprendidas

Componente IA y Hoja de Ruta Predictiva

SmartSpend está arquitectónicamente listo para la transición de analítica descriptiva a **modelos predictivos de gasto**. Python como lenguaje base del backend facilita esta evolución sin cambios de arquitectura.



Demo en Vivo

Espacio reservado para demostración en vivo o vídeo walk-through de la aplicación.

- Registro de una compra multi-artículo desde el formulario React
- Edición atómica con actualización en tiempo real del total
- Navegación por el Grid Center y gestión de entidades
- Dashboard de Power BI: filtrado por usuario, tienda y categoría

Lecciones Clave

- **Autonomía técnica:** resolución independiente de fallos de integración (CORS, FK constraints, encoding)
- **Juicio de esquema:** decisión de normalizar products antes de conectar Power BI — anticipando las necesidades analíticas
- **Arquitectura primero:** el esquema MySQL se diseñó antes que el código, garantizando que el modelo relacional guíe tanto el ORM como el Star Schema en BI
- **Calidad de datos desde el origen:** Pydantic v2 y transacciones atómicas como primera línea de defensa

 **SmartSpend no es solo un rastreador de gastos.** Es un sistema de datos relacional, un mini pipeline de ingeniería de datos, una arquitectura lista para analítica avanzada y la base para modelos predictivos con IA real.